

ENTITY RELATIONSHIP ANALYSIS

Database Architecture & Relational Mapping

July 2026

1. Introduction

Following the definition of core data entities, this document outlines the intricate relationships, cardinality, and business rules binding the system's data architecture. Establishing these relationships is crucial for maintaining data consistency, enforcing operational logic, and planning the transition from conceptual models to physical database schemas.

2. Relationship Matrix & Cardinality

The interactions between our primary entities are governed by strict cardinality constraints, ensuring that data points accurately reflect real-world business operations.

SOURCE ENTITY	RELATIONSHIP TYPE	TARGET ENTITY	CARDINALITY	DESCRIPTION
User Profile	Initiates	Transaction Record	1 : N	One user can have zero, one, or many transactions. A transaction must belong to exactly one user.
Transaction Record	Contains	Inventory Item	M : N	A transaction can contain multiple items. An item can appear in multiple distinct transactions.
Inventory Item	Belongs To	Category	N : 1	Many items can belong to a single category. An item has only one primary category.

3. Business Rules and Constraints

The defined relationships enforce several critical business rules at the database level:

- **Referential Integrity:** A `Transaction Record` cannot be created without a valid, existing `UserID`. If a user is deleted, their transactions must either be soft-deleted or anonymized to preserve financial history.
- **Inventory Depletion:** The M:N relationship resolved via the `TransactionLineItem` table acts as the trigger point for adjusting `StockQuantity` in the `Inventory Item` table.
- **Orphan Prevention:** An `Inventory Item` cannot exist without being mapped to at least one system Category, ensuring all products are discoverable via standard queries.

4. Application to Document-Based Storage (MongoDB)

While the relationships above map perfectly to a traditional SQL environment, deploying this architecture on a scalable document-based infrastructure like MongoDB requires strategic adjustments. Rather than strictly relying on complex JOIN operations, we optimize the data modeling based on query access patterns.

Implementation Strategy: Embedded vs. Referenced Documents

4.1 The User Profile (Referenced)

User Profiles will remain as distinct documents in a `users` collection. Because user data scales independently and is frequently updated, embedding transactions directly into the user document would quickly exceed the 16MB document size limit.

4.2 The Transaction Record (Embedded Line Items)

To optimize for rapid read/write operations during checkout, the associative `TransactionLineItem` data should be embedded directly as an array within the `Transaction Record` document.

```
{
  "_id": ObjectId("64f1a2b3c..."),
  "userId": ObjectId("5f8d0a..."),
  "transactionDate": "2026-07-13T09:30:00Z",
  "status": "Completed",
  "items": [
    {
      "itemId": ObjectId("7a1b..."),
      "sku": "ITEM-100",
      "quantity": 2,
      "priceAtSale": 29.99
    }
  ],
  "totalAmount": 59.98
}
```

5. Conclusion

The defined Entity-Relationship model establishes a highly organized data flow. By mapping these relationships clearly, the backend architecture can seamlessly transition between normalized relational schemas or optimized document-based collections, depending on the specific scalability needs of the Node.js API core.